02 / ALGEBRA / CLASS_17_VECTORS / EX26

Python reference guide

np.dot(), np.linalg.norm(), np.arccos(), np.degrees() and numerical accuracy.

DOCUMENT 02

Learning objectives

- Understand inputs, outputs and units of each function.
- Compute angles correctly in radians and degrees.
- Identify common mistakes before coding.

Prerequisites: Prerequisites: EX25 completed, NumPy arrays, vector components and Euclidean norm.

Reading order: 01 Theory → 02 Reference → 03 Exercise → 04 Pseudocode → Coding.

1. Tools in EX26

The Theory Guide explains the dot product. Here we focus on **how to express it in Python**, what each function accepts and returns.

```
import numpy as np

u = np.array([1.0, 2.0])
v = np.array([3.0, 4.0])
```

2. np.dot(a, b)

What it does: for two 1D arrays with equal length, it computes their dot product and returns one number.

Inputs: a, b = numeric arrays of compatible shapes; in this exercise, (2,).

Output: numeric scalar, not a two-component array.

```
a = np.array([1.0, 2.0])
b = np.array([3.0, 4.0])
product = np.dot(a, b)
print(product)  # 11.0 = 1*3 + 2*4
```

Equivalent for 1D vectors: a @ b. We will also use @ with matrices, where its meaning depends on array dimensions.

```
print(a @ b)      # 11.0
print(a * b)      # [3. 8.]  NO es producto escalar
```

3. np.linalg.norm(x)

What it does: for a 1D vector it returns its Euclidean norm by default, a nonnegative scalar magnitude.

Input: x, 1D array. **Output:** float scalar in these examples.

```
vector = np.array([5.0, 12.0])
length = np.linalg.norm(vector)
print(length)      # 13.0
```

Important: np.linalg.norm([vector]) receives a list containing the vector and treats it as a one-row matrix. Avoid extra brackets when measuring a vector.

4. np.arccos(x)

What it does: computes the angle whose cosine is x. **Input:** number between −1 and 1. **Output:** angle in **radians**.

```
theta_rad = np.arccos(0.5)
print(theta_rad)   # about 1.0472 radians
```

5. np.degrees(x) and np.clip(x, low, high)

np.degrees(x): accepts an angle in radians and returns degrees.

```
degrees = np.degrees(theta_rad)
print(degrees)   # 60.0
```

np.clip(x, -1, 1): restricts a number to [−1, 1]. It is useful for a computed cosine because tiny rounding errors can yield 1.0000000000000002.

```
cosine = 1.0000000000000002
safe_cosine = np.clip(cosine, -1.0, 1.0)
print(np.arccos(safe_cosine))  # 0.0
```

6. How to plan an angle between two vectors

Without giving the full EX26 implementation: we need the dot product, both norms, the cosine ratio and finally arccos. Check both norms are nonzero before dividing.

$$c = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

$$\theta_{\text{deg}} = \text{degrees}(\arccos(c))$$

> Do not convert to degrees before arccos. np.arccos() expects a cosine, not an angle. If either vector is zero, the angle is undefined; do not add an arbitrary tiny number to the denominator.

7. Comparisons and output formatting

Floating-point calculations can introduce tiny differences. We can compare close scalars with np.isclose; we will study tolerances in more detail later.

```
result = np.sqrt(2.0) ** 2
print(np.isclose(result, 2.0))  # True
print(f"Result: {result:.3f}")
```

8. Common mistakes and good practices

- Confusing np.dot(a, b) (scalar) with a*b (array).
- Using degrees with np.cos() without converting to radians.
- Forgetting the nonzero-vector requirement.
- Combining incompatible shapes or data without units.
- Treating a hardcoded expected result as a computed value.

9. Quick reference

np.array([x, y]) — vector (2,)

np.dot(a,b) — scalar

a @ b — scalar for 1D vectors

np.linalg.norm(a) — length

np.arccos(c) — radians

np.degrees(angle) — degrees

np.clip(c,-1,1) — bounded cosine